

Security Audit

Ithaca 2nd Updates (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	11
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Ithaca team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The Ithaca Protocol is a decentralized finance (DeFi) platform that aims to enhance liquidity provision and decentralized governance within the DeFi ecosystem. It employs advanced automated market making (AMM) mechanisms to facilitate seamless trading and liquidity management while incorporating a governance model that empowers stakeholders in decision-making processes. The protocol focuses on efficient liquidity pool management, attractive incentives for liquidity providers, and robust security measures to ensure the protection of user funds. Continuously evolving through its development roadmap, Ithaca Protocol integrates with other DeFi projects to expand its ecosystem and drive sustainable growth.

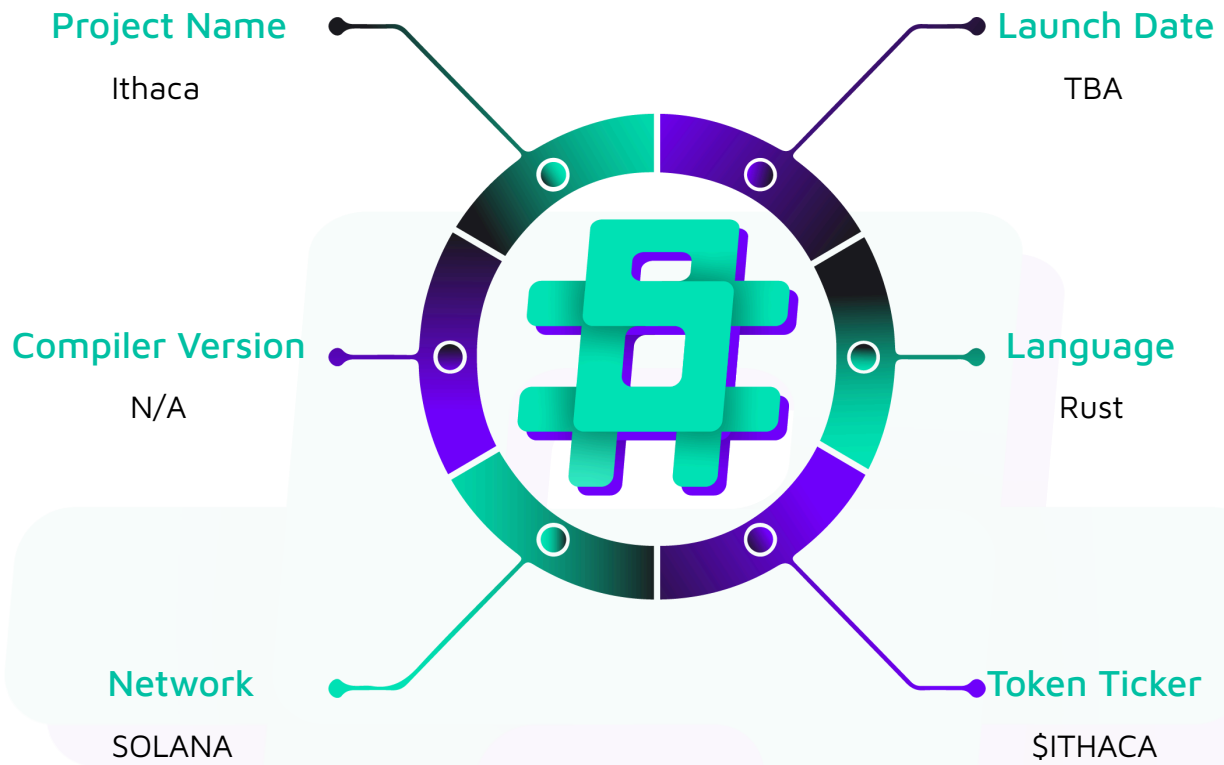
Project Name: Ithaca

Project Type: Defi

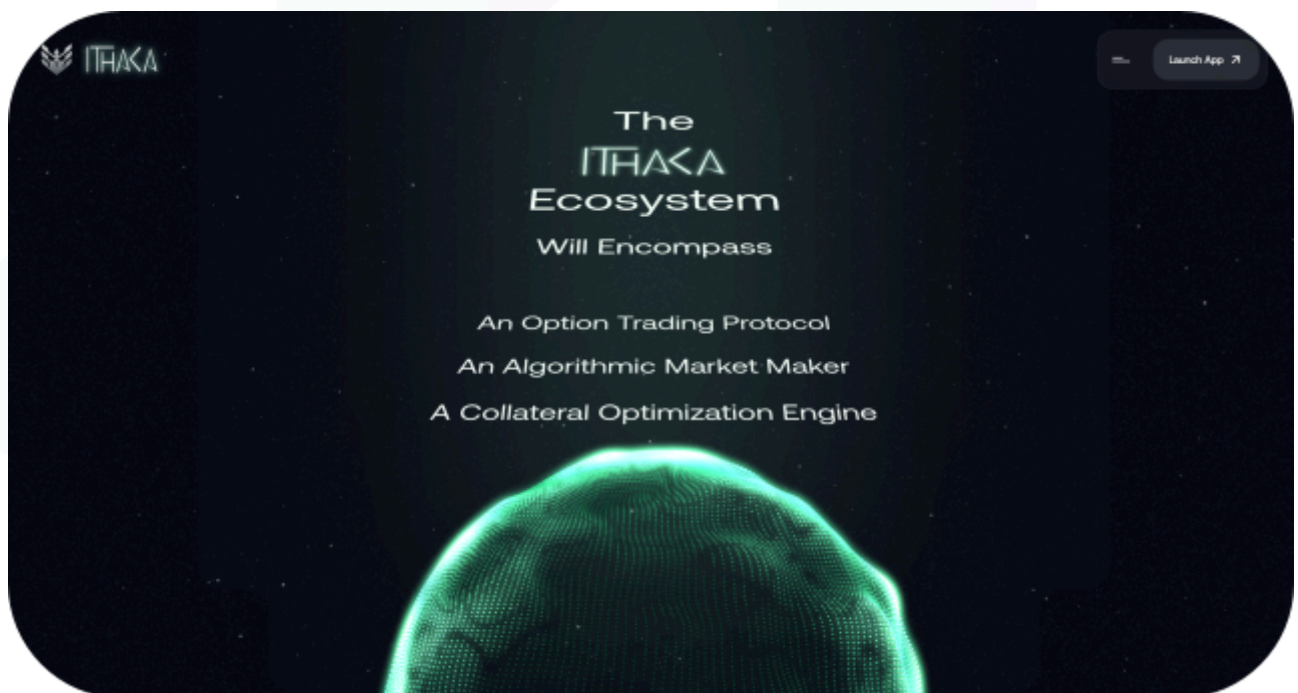
Website: <https://www.ithacaprotocol.io/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the Rust code within the Ithaca project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Ithaca Protocol Smart Contracts
Platform	Solana / Rust
Audit Date	February, 2025
Contract 1	lib.rs
Contract 1 MD5 Hash	e8f6e56b58e4d1b5eb57d0d3e19ea8af
Contract 2	error.rs
Contract 2 MD5 Hash	3bf9f7b1acd40f0dc0f3b8b5a0c6ca08
Contract 3	constants.rs
Contract 3 MD5 Hash	5eeda70c4cbdee81c7f1323bde7abbb2
Contract 4	access_controller_state.rs
Contract 4 MD5 Hash	0ec55b2a2d53d6bd25b5af232f6c98f2
Contract 5	fundlock_state.rs
Contract 5 MD5 Hash	b8afe32f8b9b22f99ab63d8c81cf2e14
Contract 6	ledger_state.rs
Contract 6 MD5 Hash	a5bb29b62e2f09b80db77d31b7d45ddf
Contract 7	token_validator_state.rs
Contract 7 MD5 Hash	2cfa7f08eb34ffba43df95e60a3eb3a4
Contract 8	check_role.rs
Contract 8 MD5 Hash	08a05c751daa8493d31e6a352e9cfc64
Contract 9	grant_role.rs

Contract 9 MD5 Hash	733df5825ae761ec40c98a5048b38493
Contract 10	init_access_controller.rs
Contract 10 MD5 Hash	a2e7f2fb95266ad1ca0c50f55ff97be1
Contract 11	renounce_role.rs
Contract 11 MD5 Hash	d7b4d8fc5c9c45ae37c5841a7ac2ef1a
Contract 12	deposit_fundlock.rs
Contract 12 MD5 Hash	1dec49de54c7b36290ccffcbe3d63f9a
Contract 13	init_fundlock.rs
Contract 13 MD5 Hash	3c1d8b4af38a6fafb0da232f40e3e301
Contract 14	release_fundlock.rs
Contract 14 MD5 Hash	8d87f49f1a8b8b22e31cf8d0afcf9cdf
Contract 15	withdraw_fundlock.rs
Contract 15 MD5 Hash	1f5a0fdfef2d40dc3b07fee3a70e59b9
Contract 16	deposit_kamino.rs
Contract 16 MD5 Hash	91ae1e027a5661edc85c9cfb9cc6f829
Contract 17	reedem_kamino.rs
Contract 17 MD5 Hash	a0bd8dae5463e6af78dc06d8560034d2
Contract 18	update_fund_movements.rs
Contract 18 MD5 Hash	ecc2ed36cd23cfccddb4f14da3f31fd
Contract 19	init_ledger.rs
Contract 19 MD5 Hash	1f2b1f5b05f9c8c1cd8e87a1ef6f75db
Contract 20	add_token_to_whitelist.rs
Contract 20 MD5 Hash	11c80e4eecccafcf4d4d80c4033fb1e1
Contract 21	init_token_validator.rs
Contract 21 MD5 Hash	a35a52dde4a347aead5c91c22a49fa0d
Contract 22	remove_token_from_whitelist.rs

Contract 22 MD5 Hash	54c2bba11b1def2f4b6d53ac67d6b4df
Contract 23	init_client_data.rs
Contract 23 MD5 Hash	acb518ec95ca41c8b24f1f1ca38d3ee0
Contract 24	init_fundlock_vault.rs
Contract 24 MD5 Hash	3c1d8b4af38a6fafb0da232f40e3e301
Contract 25	mod.rs (instructions directory)
Contract 25 MD5 Hash	6f1bac73de03e6add7be9afb6db0f8c1
Contract 26	set_activate_strategy.rs
Contract 26 MD5 Hash	3bd0cb7b764b40f40c1b9de3aa25e90a
Contract 27	set_intervals.rs
Contract 27 MD5 Hash	45c0dd56bb8b92608dd95dc3ca3b4eb4
Contract 28	mod.rs (ledger directory)
Contract 28 MD5 Hash	c2d968ee07dbb3a1cd0f7ce84e9e9c0e
Contract 29	init_strategy_supplying_vault.rs
Contract 29 MD5 Hash	246ecbcd3ca7b4a0d3a193c693d9e38e
Contract 30	init_strategy_yield_vault.rs
Contract 30 MD5 Hash	9e20c1e10ba318cc07ca1ad4e1f7ee59
Contract 31	init_strategy.rs
Contract 31 MD5 Hash	8c4b90dd12edc99c5551db14cb5a7f85
Contract 32	mod.rs (strategies directory)
Contract 32 MD5 Hash	5a03992e51d25dade75d4c9bbdf118d1
Contract 33	set_max_managing_ratio.rs
Contract 33 MD5 Hash	5c77c842e8f41ff22bbe12ccc7a1fc3d
Contract 34	mod.rs (token_validator directory)
Contract 34 MD5 Hash	3d1fd2def9fb0acc70cb6d25d51a7c83
Contract 35	mod.rs (state directory)

Contract 35 MD5 Hash	OdeOdOf9e93d68a5b9aed0a8f387df4e
Contract 36	strategies_state.rs
Contract 36 MD5 Hash	0fb2b35d1c0df64d9ac9c5f0c67f5feb
Contract 37	mod.rs (utils directory)
Contract 37 MD5 Hash	c5bb41d0fe5f8ad9bdc9d86a538ec85c
Contract 38	strategies_utils.rs
Contract 38 MD5 Hash	eb30e09d67de4bc2273b02b63d7de2fe
GitHub Commit Hash	048783219eea12dd9302053c03b4d42c01feea94

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This audit scope involves the smart contracts of the Ithaca project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Ithaca project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `update_fund_movements` - Incorrect Reserve Data Usage in Fund Movement Strategy

Description

In the `update_fund_movements` function, the code incorrectly uses data from the underlying token reserve when processing strike token strategies. When calculating exchange rates, available liquidity, and yield distribution for strike tokens with enabled strategies, the function mistakenly references the underlying reserve data instead of the strike reserve data.

Vulnerability Details

The issue occurs in the `update_fund_movements.rs` file during the processing of strike token balances. When a strike token has its strategy enabled, the function retrieves reserve data from the wrong source:

```
if client_balance_strike.strategy_enabled {  
  
    (mint_total_supply_strike, total_liquidity_strike) = self.get_data_from_reserve(  
  
        self.underlying_reserve.as_ref()  
  
        .expect("msg: underlying_reserve not found")  
  
        .try_borrow_data()  
  
        .expect("msg: failed to borrow data")  
  
        .as_ref()  
  
    );  
  
    // ... rest of strike token processing code  
  
}
```

Impact

Strike token strategies were previously using exchange rates derived from incorrect reserve data, leading to miscalculations in the valuation of yield-bearing assets. As a result, users could receive incorrect yield amounts, either causing financial losses or granting unearned gains. Additionally, if the underlying reserve's exchange rate significantly differed from the strike reserve's, the system could overestimate available funds, increasing the risk of insolvency.

Recommendation

Replace the incorrect reference to the underlying reserve with the strike reserve:

```
if client_balance_strike.strategy_enabled {  
  
    (mint_total_supply_strike, total_liquidity_strike) = self.get_data_from_reserve(  
        self.strike_reserve.as_ref()  
  
        .expect("msg: strike_reserve not found")  
  
        .try_borrow_data()  
  
        .expect("msg: failed to borrow data")  
  
        .as_ref()  
  
    );  
  
    // ... rest of code  
}
```

Note

The Ithaca team reported to Hashlock that this vulnerability has been resolved by replacing the use of `underlying_reserve` with `strike_reserve`. The team discovered that during the strike balance iteration within the update process, incorrect data was being utilized. With this change, the fund movement strategy now correctly references the strike reserve, ensuring accurate and reliable data usage throughout the operation.

Status

Resolved



Medium

[M-01] Multiple Contracts - Potential Over/Under flow due to Systemic Use of Unsafe Arithmetic Operations in Protocol State Changes

Description

The protocol extensively uses direct arithmetic operators (+, -) for state changes rather than checked arithmetic operations. In a Solana program context, these operations can cause program panics on overflow/underflow instead of returning handleable errors, consuming compute units and transaction fees while providing an unknown panic at runtime

Vulnerability Details

This pattern appears throughout critical protocol functions. Here are representative examples:

```
// In `deposit_kamino.rs`  
  
// Unsafe arithmetic that can panic  
self.client_balance.collateral_amount += deposit_collateral_amount;  
  
// In `withdraw_fundlock.rs`:  
  
// Multiple instances of unsafe operations  
self.withdrawals.active_withdrawals_amount += amount;  
self.client_balance.amount -= amount;  
  
// In `update_fund_movements.rs`:  
  
// Unsafe type casting and arithmetic  
client_balance_underlying.amount = (client_balance_underlying.amount as i64 +  
_change_in_underlying_balance) as u64;
```

Impact

When these operations overflow or underflow, the program panics instead of returning a proper error that could be handled gracefully in a better way.

Recommendation

Replace unchecked arithmetic with safe operations throughout the protocol

Note

The Ithaca team communicated that the vulnerability has been resolved by replacing all potential overflow operations with safer, checked arithmetic alternatives. The team emphasized that while such arithmetic flaws could theoretically lead to critical errors during protocol state transitions, no actual exploit had been observed. This proactive measure not only mitigates the risk of overflow or underflow issues but also reinforces the overall integrity and robustness of the protocol.

Status

Resolved

[M-02] `init_access_controller` - Access Control Could be Reinitialized to a Parallel State

Description

The protocol's initialization mechanism in `init_access_controller` allows creation of multiple valid instances due to admin-dependent PDA derivation. The access controller PDA uses the admin's public key as a seed, enabling parallel protocol states rather than enforcing a singleton design.

Vulnerability Details

The access controller PDA derivation includes the admin's public key as a seed:

```
#[account(  
  init,  
  payer = admin,
```

```

seeds = [b"access_controller".as_ref(), admin.key().as_ref()],

bump,

space = AccessController::INIT_SPACE,

)]

pub access_controller: Account<'info, AccessController>

```

This allows different admins to create unique but valid protocol instances, each with their own governance and state tree.

Impact

1. Multiple valid protocol instances can exist simultaneously
2. Each instance has full protocol functionality and its own governance
3. No mechanism to identify the official protocol instance
4. Creates risks for protocol integrations and user interactions

Recommendation

Add a global state account to track initialization status and make sure that the contract cannot be reinitialized.

Note

The Ithaca team advised Hashlock that it has been addressed by hardcoding the expected admin before deployment. While parallel states are a common pattern in Solana programs and are not typically classified as a medium-level vulnerability, this additional measure was implemented purely as a safety precaution. This change ensures that only the designated administrator can reinitialize the access controller, thereby reinforcing the overall integrity of the system.

Status

Resolved

[M-03] deposit, withdraw, set_activate_strategy, release_fundlock - Yield Calculation Validation Bypass

Description

In the yield-bearing strategy implementation, when the protocol calculates that no fund adjustment is needed (when amount == 0), it skips the CPI calls to the Kamino lending program, inadvertently bypassing validation of the `kamino_reserve` account. This allows an attacker to provide a malicious `kamino_reserve` account that can manipulate yield calculations without being properly validated.

The vulnerability stems from the protocol's architectural flow where data from the `kamino_reserve` account is read and used in yield calculations before the account is fully validated. The validation happens conditionally only when fund adjustments are needed, creating a security gap when no adjustments are required.

Vulnerability Details

The core issue revolves around how these functions interact with the Kamino lending protocol to calculate and distribute yield:

The issue is explained more deeply in the comments below with respect to the codebase:

```
// In functions like deposit_fundlock.rs, withdraw_fundlock.rs, etc.  
  
// First, data is read from kamino_reserve without full validation  
if strategy_enabled {  
    (mint_total_supply, total_liquidity) = self.get_data_from_reserve()?;  
  
    // This data is used in yield calculations that affect user balances  
    let distribute_yield_results = distribute_yield(  
        // ... various parameters including mint_total_supply and total_liquidity ...  
    );  
};
```

```

// User balances are updated based on these calculations

token_strategy.yield_index = distribute_yield_results.0;

token_strategy.last_generated_yield = distribute_yield_results.1;

// ... more balance updates ...


// Later, decide if adjustment is needed

let adjust_fund_result = adjust_fund(

    // ... various parameters ...

);

let amount = adjust_fund_result.1;

// Only if adjustment is needed does the protocol validate kamino_reserve via CPI

if amount == 0 {

    msg!("No funds to adjust");

} else {

    if adjust_fund_result.0 == true {

        self.utilize_funds(amount).expect("Failed to utilize funds");

    }

    if adjust_fund_result.0 == false {

        self.unutilize(amount, mint_total_supply, total_liquidity)

            .expect("Failed to unutilize funds");

    }

}

}

```

The root cause is in the `get_data_from_reserve` function, which directly reads and deserializes data from `kamino_reserve` without proper validation

Impact

1. An attacker can manipulate calculations to receive a disproportionately high share of yield at the expense of other users.
2. The protocol's internal accounting would be corrupted, creating an uneven distribution of rewards.

Recommendation

1. Validate the `kamino_reserve` account at the beginning of the function execution, before any data is read or used
2. Clearly separate the validation logic from the data usage to ensure validation always happens first.

Note

Ithaca notified Hashlock that the vulnerability identified has been fully addressed through a series of targeted modifications. Specifically, the team added a new "token_strategy" field to the `FundlockVaultState` struct and implemented the `set_strategy_for_vault.rs` method to initialize this field appropriately. Additionally, the seeds for the `token_strategy` account were updated by incorporating the `kamino_market` key, and further validations were introduced to detect and prevent any mismatches between the provided `token_strategy` and `kamino_market` values—this includes constraints added in `set_active_strategy.rs`. Moreover, checks have been implemented to ensure that the `kamino_reserve`'s `lending_market` matches the expected configuration. These comprehensive changes ensure that the kamino market and the corresponding supplying token key are explicitly set as seeds for the token strategy, thereby constraining data and reinforcing safe retrieval before CPI calls.

Status

Resolved

Low

[L-01] strategies_utils - Missing check for zero total_liquidity in exchange rate calculation

Description

The `latest_exchange_rate` function in `strategies_utils.rs` performs division by `total_liquidity` without first checking if this value is zero. This oversight can lead to a runtime panic due to division by zero, causing transaction failures during critical operations.

```
pub fn latest_exchange_rate(total_liquidity: Fraction, mint_total_supply: u64) -> u128 {  
    // Missing check: what if total_liquidity is zero?  
  
    let fraction_rate = Fraction::from(mint_total_supply) / total_liquidity;  
  
    let rate: u128 = fraction_rate  
        .checked_mul(Fraction::from_num(EXCHANGE_MULTIPLIER))  
        .expect("Failed to multiply fraction rate with exchange multiplier")  
        .checked_floor()  
        .expect("Failed to floor the latest exchange rate")  
        .checked_to_num()  
        .expect("Failed to convert the latest exchange rate to u64");  
  
    msg!("Latest Exchange Rate: {}", rate);  
  
    rate  
}
```

Impact

If `total_liquidity` ever becomes zero (which is possible during market initialization or in low liquidity scenarios), any transaction that depends on exchange rate calculations will fail.

Recommendation

Add an explicit check for zero `total_liquidity` and handle it appropriately

Note

The Ithaca team instructed that the vulnerability identified has been addressed by implementing an explicit check that throws an error if the total liquidity is zero. While the likelihood of encountering a Kamino reserve with zero liquidity is exceedingly low, the team added this safeguard as a precautionary measure to enhance system robustness and ensure accurate exchange rate calculations.

Status

Resolved

[L-02] strategies_utils - Missing String Length Prefix in Role Struct Space Calculation

Description

The space calculation for the Role struct does not account for the 4-byte prefix that Borsh serialization includes for strings:

```
// Current implementation

impl Space for Role {

    const INIT_SPACE: usize = 8 + // account discriminator

    32 + // role string (expected max size of 32 bytes)

    8 + // member count

    1; // bump

}

// Struct definition

#[account]

pub struct Role {
```

```
pub role: String,

pub member_count: u64,

pub bump: u8,

}
```

Impact

When strings are serialized in Anchor (using Borsh), a 4-byte length prefix is added before the string data. This prefix is missing from the size calculation, resulting in a 4-byte underallocation.

As a consequence, if a role name approaches the expected maximum size, the serialized data could exceed the allocated space, potentially causing runtime failures during account creation or updates.

Recommendation

Update the space calculations to accurately reflect the data types:

```
// For Withdrawals

impl Space for Withdrawals {

    const INIT_SPACE: usize = 8 + // account discriminator

    4 + (ALLOWED_WITHDRAWAL_LIMIT * 16) + // withdrawals limited to 5 per client

    8 + // active withdrawals amount

    32 + // client balance pda

    1; // bump - corrected to 1 byte

}

// For Role

impl Space for Role {

    const INIT_SPACE: usize = 8 + // account discriminator

    4 + 32 + // string length prefix (4 bytes) + role string (max 32 bytes)

    8 + // member count

}
```

```
1; // bump  
}
```

Note

The Ithaca team informed Hashlock that the vulnerability was addressed by incorporating a 4-byte prefix to the Role field. While the Role string enum is not anticipated to exceed 32 bytes, this modification ensures that the maximum allocation is correctly set, preventing any miscalculations where the maximum might otherwise be limited to 28 bytes.

Status

Resolved

[L-03] fundlock_state - Incorrect Size Allocation for bump in Withdrawals Struct

Description

The implementation for the `Withdrawals` struct incorrectly allocates 8 bytes for the `bump` field in the `Withdrawals` struct when only 1 byte is required, resulting in inefficient on-chain storage utilization.

The space calculation for the `Withdrawals` struct allocates 8 bytes for the `bump` field, despite this field being defined as a `u8`, which only requires 1 byte:

```
// Current implementation

impl Space for Withdrawals {

    const INIT_SPACE: usize = 8 + // account discriminator

    4 + (ALLOWED_WITHDRAWAL_LIMIT * 16) + // withdrawals limited to 5 per client

    8 + // active withdrawals amount

    32 + // client balance pda

    8; // bump - allocating 8 bytes for a u8

}

// Struct definition

#[account]

pub struct Withdrawals {

    pub withdrawal_queue: Vec<Box<WithdrawalState>>,

    pub active_withdrawals_amount: u64,

    pub client: Pubkey,

    pub bump: u8, // Only needs 1 byte

}
```

Impact

This mismatch results in an overallocation of 7 bytes per Withdrawals account. When the protocol has many users, each with multiple withdrawals, this overallocation accumulates, unnecessarily increasing the overall storage.

Recommendation

Update the space calculation to accurately reflect the data type of the bump field:

```
// Corrected implementation

impl Space for Withdrawals {

    const INIT_SPACE: usize = 8 + // account discriminator

    4 + (ALLOWED_WITHDRAWAL_LIMIT * 16) + // withdrawals limited to 5 per client

    8 + // active withdrawals amount

    32 + // client balance pda

    1; // bump - corrected to 1 byte

}
```

Note

The Ithaca team informed that the issue has been resolved by adjusting the bump size to precisely 1 byte. Although the report noted that the extra 7 bytes in the fundlock state would not have caused any operational issues, the team emphasized the importance of precise memory allocation to avoid allocating unnecessary space. This resolution underscores Ithaca's commitment to adhering to best practices in memory management and ensuring optimal account configuration.

Status

Resolved

QA

[Q-01] `add_token_to_whitelist` - Unclear Error Handling for Already-Whitelisted Tokens

Description

When an administrator attempts to whitelist a token that has already been whitelisted, the transaction fails with a generic Solana runtime error message about account creation failure. This occurs because the instruction uses the `init` constraint without proper error handling for existing accounts, resulting in confusing error messages that don't clearly indicate the actual issue.

Recommendation

Implement Clear Error Handling

Note

The Ithaca team confirms Hashlock that this finding has been carefully reviewed. Given the team's clear understanding of the tokens in use, they determined that the existing error handling is sufficient, and no additional measures were necessary.

Status

Acknowledged

[Q-02] `Multiple Contracts` -Missing Event Emissions for Critical State Changes

Description

While the protocol implements some event logging through `msg!` macros, several critical state changes lack proper event emissions. Comprehensive event logging is crucial for protocol monitoring, debugging, and integration with external systems.

Recommendation

Add comprehensive event logging for critical state changes

Note

The Ithaca team emphasized that ensuring complete and consistent event logging is an integral part of their responsibilities. To that end, they have implemented and will implement measures to guarantee that event logs align with the expectations of the backend, subgraph, and related systems—both now and in the future. This approach ensures that all critical state changes are reliably captured and available for monitoring and analysis.

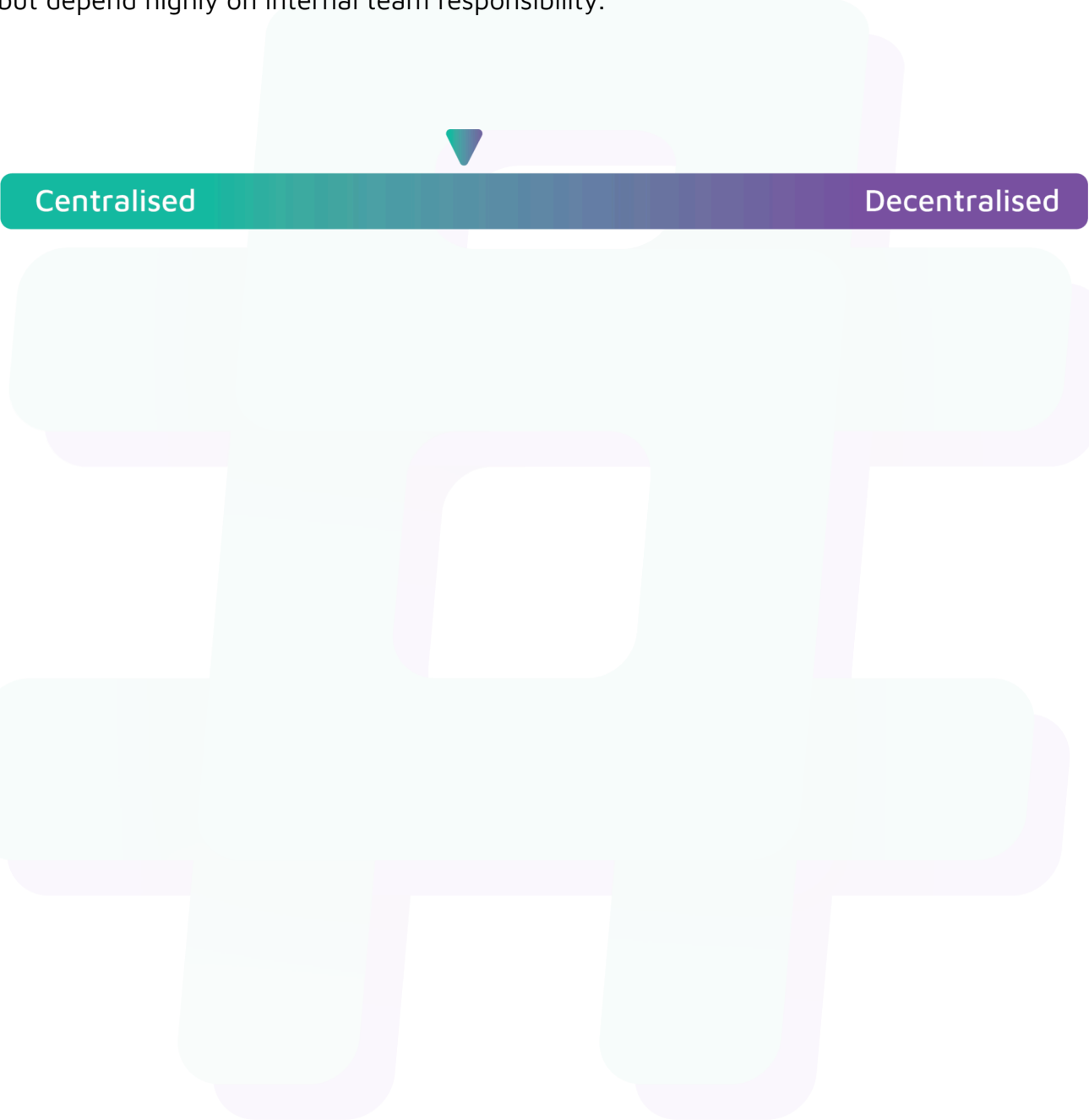
Status

Acknowledged

Centralisation

The Ithaca project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Ithaca project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.